

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



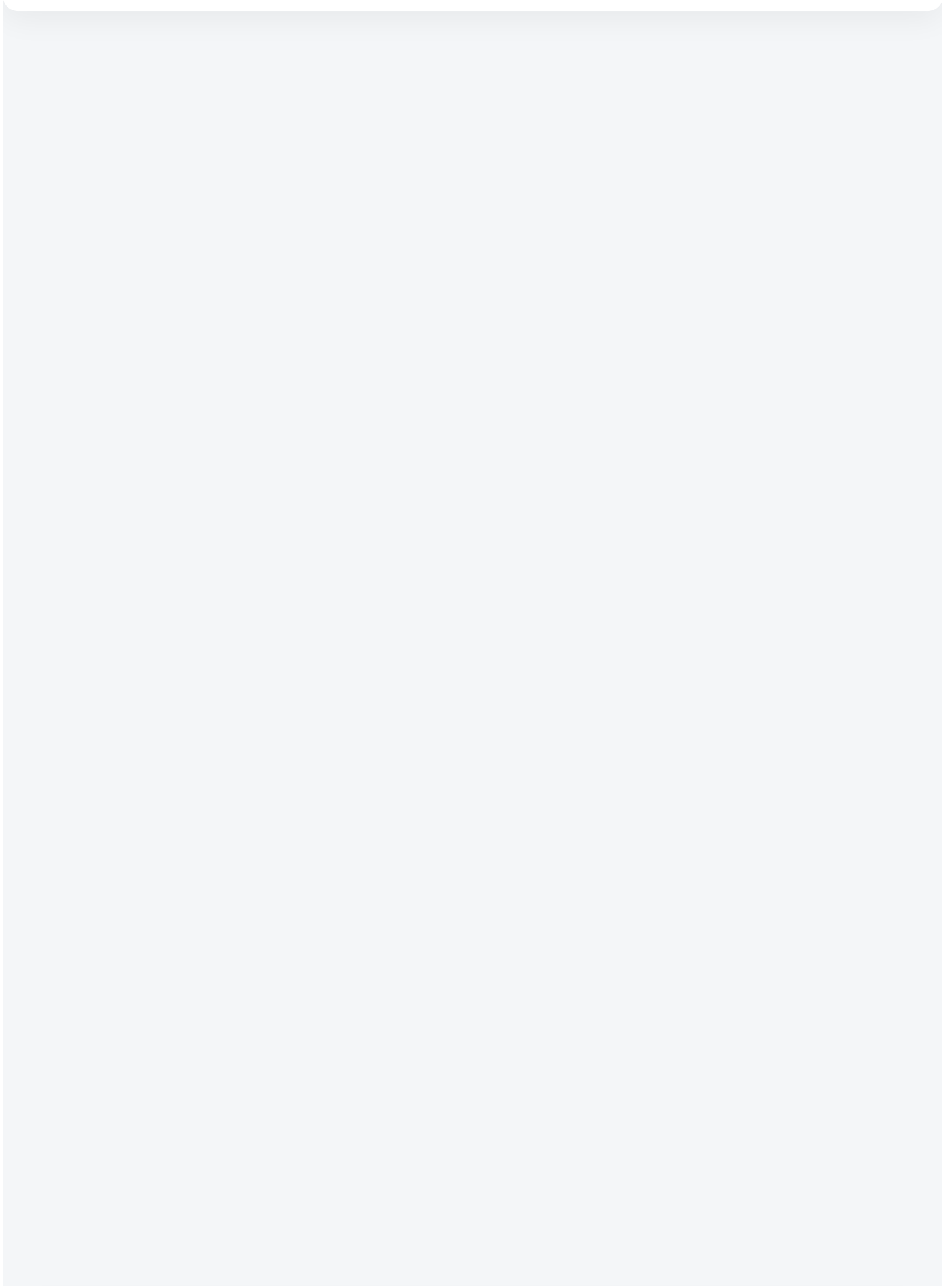
Rapport DevOps — AWS CodePipeline

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**



Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)

1. Objectifs et contexte

AWS CodePipeline est un service d'orchestration CI/CD permettant d'enchaîner des actions de récupération du code, de compilation, de test, d'approbation et de déploiement. L'objectif de ce travail est de construire un flux reproductible allant du dépôt source jusqu'à l'environnement cible.

- Comprendre la structure **pipeline** → **stages** → **actions** → **artefacts**.
- Connecter CodeCommit, CodeBuild et CodeDeploy.
- Automatiser le déclenchement à partir d'un changement du dépôt.
- Ajouter une approbation manuelle avant la production.
- Contrôler les exécutions, les artefacts et les erreurs.

Principe : CodePipeline orchestre les services ; il ne remplace pas CodeBuild pour compiler ni CodeDeploy pour déployer.

2. Prérequis

- Un compte AWS et une région de travail, par exemple `us-east-1`.
- AWS CLI configurée et autorisée à interroger les services nécessaires.
- Un dépôt CodeCommit contenant l'application et, si nécessaire, `buildspec.yml` et `appspec.yml`.
- Un projet CodeBuild fonctionnel produisant un artefact de sortie.
- Une application et un groupe de déploiement CodeDeploy opérationnels.
- Un bucket S3 destiné au stockage des artefacts du pipeline.
- Des rôles IAM séparés et dimensionnés selon le principe du moindre privilège.

```
# Vérifier l'identité AWS utilisée par la CLI aws sts get-caller-identity # Vérifier
la région par défaut aws configure get region # Vérifier les ressources préparées aws
codecommit get-repository --repository-name mon-app aws codebuild batch-get-projects
--names mon-app-build aws deploy get-application --application-name mon-app
```

Attention : les noms `mon-app`, `mon-app-build` et les ARN utilisés dans ce rapport sont des exemples à adapter à votre compte.

3. Concepts et architecture

Développeur | git push ▼ CodeCommit —EventBridge→ CodePipeline | | Source → artefact source | | Build → CodeBuild → artefact applicatif | | Approval (optionnel) | | Deploy → CodeDeploy → EC2 / autre cible Artefacts : S3 + chiffrement KMS

Élément	Responsabilité	Exemple
Pipeline	Orchestration globale	mon-app-pipeline
Stage	Regroupe des actions d'une phase	Source, Build, Deploy
Action	Travail exécuté dans un stage	CodeBuild, CodeDeploy, Manual
Artefact	Données transmises entre actions	source.zip, application.zip
EventBridge	Détection événementielle des changements CodeCommit	Push sur main
IAM	Autorisation des services	Service role CodePipeline

Modèle d'action : AWS documente six catégories principales : Source, Build, Test, Deploy, Approval et Invoke. Les actions peuvent être séquentielles ou parallèles selon la configuration du stage. [CiteTurn0search3](#)

4. Installation et configuration

4.1 Bucket S3 d'artefacts

```
export AWS_REGION=us-east-1 export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text) export ARTIFACT_BUCKET=mon-app-codepipeline-artifacts-$ACCOUNT_ID aws s3api create-bucket --bucket "$ARTIFACT_BUCKET" --region "$AWS_REGION" # Pour une région différente de us-east-1, préciser LocationConstraint. # Exemple : --create-bucket-configuration LocationConstraint=eu-west-3
```

4.2 Rôle de service CodePipeline

Le rôle de service est assumé par CodePipeline et doit autoriser uniquement les opérations nécessaires sur S3, CodeCommit, CodeBuild, CodeDeploy et, si activé, SNS/KMS. En pratique, la console peut créer le rôle ; en CLI, le rôle et sa policy doivent être préparés explicitement.

4.3 Détection des changements

Pour une source CodeCommit, la méthode recommandée est événementielle avec **Amazon EventBridge**. Lorsqu'un pipeline est créé ou modifié depuis la console avec CodeCommit, la règle EventBridge est créée automatiquement. Avec la CLI, les ressources de détection doivent être configurées séparément. `□cite□turn0search5□turn0search10□`

 Création du pipeline CodePipeline

5. Réalisation pratique

5.1 Création du pipeline

```
# Décrire le pipeline à partir d'un fichier JSON aws codepipeline create-pipeline --  
cli-input-json file://pipeline-config.json # Exemple minimal de structure (à  
compléter avec les configurations réelles) { "pipeline": { "name": "mon-app-  
pipeline", "roleArn": "arn:aws:iam::123456789012:role/CodePipelineServiceRole",  
"artifactStore": { "type": "S3", "location": "mon-app-codepipeline-artifacts-  
123456789012" }, "stages": [] } }
```

5.2 Stage Source

Le stage Source récupère une révision du dépôt et produit un artefact de sortie. Avec CodeCommit, l'action utilise le dépôt et la branche configurés.

5.3 Stage Build

CodeBuild reçoit l'artefact source en entrée, exécute le `buildspec.yml`, les tests et la génération du livrable, puis produit l'artefact de sortie.

5.4 Approbation

Une action d'approbation peut arrêter le pipeline avant la production. Elle doit être placée après les contrôles nécessaires et avant le déploiement critique. Une absence de décision pendant sept jours entraîne l'échec de l'action. [Citer la recherche](#)

5.5 Stage Deploy

CodeDeploy consomme l'artefact et réalise le déploiement selon la stratégie configurée. Pour CodeDeploy, l'action de déploiement attend un artefact d'entrée. [Citer la recherche](#)

```
# Lancer manuellement une exécution aws codepipeline start-pipeline-execution --name  
mon-app-pipeline # Voir l'état des stages/actions aws codepipeline get-pipeline-state  
--name mon-app-pipeline
```

6. Commandes et explications

Lire la définition complète du pipeline `aws codepipeline get-pipeline --name mon-app-pipeline` # Lister les pipelines `aws codepipeline list-pipelines` # Consulter les exécutions `aws codepipeline list-pipeline-executions --pipeline-name mon-app-pipeline` # Consulter les détails d'une exécution `aws codepipeline get-pipeline-execution \ --pipeline-name mon-app-pipeline \ --pipeline-execution-id EXECUTION_ID` # Arrêter une exécution en cours `aws codepipeline stop-pipeline-execution \ --pipeline-name mon-app-pipeline \ --pipeline-execution-id EXECUTION_ID \ --abandon`

Commande	Utilité
<code>get-pipeline</code>	Inspecter la définition et les stages.
<code>get-pipeline-state</code>	Identifier le stage/action actuellement en erreur.
<code>list-pipeline-executions</code>	Analyser l'historique des exécutions.
<code>start-pipeline-execution</code>	Déclencher un test manuel.
<code>stop-pipeline-execution</code>	Arrêter une exécution problématique.

Point important : une définition CLI CodeCommit ne suffit pas à elle seule pour obtenir un déclenchement événementiel ; avec la CLI, la règle EventBridge et ses autorisations doivent aussi être configurées. [citerturn0search5](#)

7. Vérifications et tests

1. Vérifier que le pipeline existe et que les stages sont correctement nommés.
2. Déclencher une exécution manuelle et observer chaque action.
3. Modifier le dépôt CodeCommit puis effectuer un `git push`.
4. Vérifier que l'événement EventBridge déclenche une nouvelle exécution.
5. Contrôler que CodeBuild reçoit l'artefact source et produit le livrable attendu.
6. Vérifier que CodeDeploy reçoit exactement l'artefact attendu.
7. En cas d'approbation, vérifier que le pipeline s'arrête puis reprend après validation.

```
# Contrôles CLI
aws codepipeline get-pipeline --name mon-app-pipeline
aws codepipeline get-pipeline-state --name mon-app-pipeline
aws codepipeline list-pipeline-executions --pipeline-name mon-app-pipeline
# Contrôler la présence de l'artefact dans S3
aws s3 ls s3://$ARTIFACT_BUCKET/ --recursive
```

Critère de réussite : une modification du code doit parcourir Source → Build → éventuelle Approval → Deploy, avec un artefact valide à chaque transition.

8. Dépannage

Symptôme	Cause probable	Action corrective
Le pipeline ne démarre pas après un push	EventBridge absent ou mal autorisé	Vérifier la règle, sa cible et les permissions ; éviter de mélanger inutilement polling et EventBridge.
Source réussie, Build échoue	buildspec ou environnement CodeBuild	Consulter les logs CodeBuild et tester le build indépendamment.
Deploy échoue	appspec, agent ou IAM	Contrôler CodeDeploy, les hooks, l'agent et les rôles.
Accès S3 refusé	Rôle IAM ou KMS incomplet	Vérifier les permissions du rôle et la clé KMS utilisée.
Approval bloquée	Absence de permission de validation	Attribuer les permissions d'approbation nécessaires à l'identité concernée.

```
# Vérifier l'état du pipeline aws codepipeline get-pipeline-state --name mon-app-pipeline # Vérifier les règles EventBridge aws events list-rules --name-prefix codepipeline # Examiner les événements récents dans la console EventBridge/CloudWatch
```

9. Sécurité et bonnes pratiques

- Utiliser des rôles IAM dédiés plutôt que des clés d'accès statiques dans les scripts.
- Appliquer le principe du moindre privilège au rôle CodePipeline et aux rôles des services aval.
- Chiffrer les artefacts S3 avec AWS KMS lorsque le niveau de confidentialité l'exige.
- Ne jamais placer de mots de passe, tokens ou clés privées dans CodeCommit, les buildspecs ou les artefacts.
- Utiliser Secrets Manager ou Parameter Store pour les secrets applicatifs.
- Séparer les environnements développement, staging et production.
- Placer une approbation humaine avant les déploiements de production sensibles.
- Conserver les journaux nécessaires à l'audit et surveiller les échecs.
- Préférer EventBridge pour la détection des changements CodeCommit plutôt que le polling lorsqu'il est disponible. [\[cite\]turn0search10](#)

Erreur critique : ne pas donner à un rôle de pipeline des permissions administrateur globales simplement pour « faire fonctionner » le pipeline. Identifier les permissions réellement nécessaires à chaque action.

Approbation et IAM

Les personnes autorisées à approuver ou rejeter une action doivent disposer des permissions CodePipeline correspondantes. AWS fournit notamment la policy [AWSCodePipelineApproverAccess](#), mais une policy limitée aux ressources nécessaires est préférable lorsque le contexte le permet. [\[cite\]turn0search2](#)

10. Checklist

Vérification finale

- ☐ Région et identité AWS vérifiées
- ☐ CodeCommit contient le code source
- ☐ CodeBuild compile, teste et produit un artefact
- ☐ CodeDeploy et son groupe de déploiement sont opérationnels
- ☐ Bucket S3 d'artefacts et chiffrement vérifiés
- ☐ EventBridge déclenche le pipeline après un push
- ☐ Source → Build → Deploy fonctionne de bout en bout
- ☐ Approbation manuelle testée si elle est utilisée
- ☐ Permissions IAM réduites au besoin réel
- ☐ Dépannage et vérifications CLI maîtrisés

11. Conclusion

Ce travail montre comment AWS CodePipeline peut servir d'orchestrateur central d'une chaîne CI/CD. Le pipeline relie le contrôle de version, la construction, les contrôles qualité et le déploiement tout en faisant circuler des artefacts entre les différentes actions.

- **Source** : CodeCommit ou un autre fournisseur compatible.
- **Build/Test** : CodeBuild exécute les traitements automatisés.
- **Approval** : une validation humaine peut protéger la production.
- **Deploy** : CodeDeploy ou un autre fournisseur réalise la livraison.
- **Événements** : EventBridge permet une détection événementielle des changements CodeCommit.
- **Gouvernance** : IAM, KMS, logs et séparation des environnements renforcent la sécurité.

Perspective : cette architecture peut ensuite évoluer vers ECS/EKS, des déploiements Blue/Green, des tests automatisés supplémentaires, des contrôles de sécurité et des pipelines multi-environnements.